Lesson Notes

MODULE 8 | LESSON 4

REINFORCEMENT LEARNING IN REAL LIFE

Reading Time	30 minutes
Prior Knowledge	Reinforcement Learning
Keywords	Value-based algorithms, Policy-based algorithms, VPG (Vanilla Policy Gradient), DDPG (Deep Deterministic Policy Gradient), TD3 (Twin Delayed DDPG), PPO (Proximal Policy Optimization), SAC (Soft Actor-Critic), Advantage Actor Critic (A2C), Asynchronous Advantage Actor-Critic (A3C), TRPO (Trust Region Policy Optimization)

In the last lesson, we introduced reinforcement learning as if it was your first exposure, though it should not have been given your coursework in Machine Learning and Stochastic Modeling. To that end, the lesson notes and required reading provided a theoretical foundation for deep reinforcement learning in its application to portfolio management.

In this lesson, we focus more on the application of the theory. We introduce a Python library FinRL (example code in a GitHub repo as well as very accessible documentation) that makes reinforcement learning for portfolio construction as simple as possible, with very practical and yet versatile defaults. To complement this, we read about a recently published DRL model that uses this FinRL library and is compared to some familiar baseline algorithms, which we can now take the time to understand better.

1. The FinRL Library

We hope you find the introductory notebooks extremely user-friendly. They do a nice job of walking you through the steps to train DRL models and compare their performance.

1. Install, load, and import the requisite libraries; create folders in your directory

Note: When you use the notebook from the Github repo that the first block of code should be replaced. Use this: `!pip install git+https://github.com/AI4Finance-Foundation/FinRL.git`

Instead of this: `!pip install git+https://github.com/AI4Finance-Foundation/FinRL-Library.git`

1. Download and inspect the market data

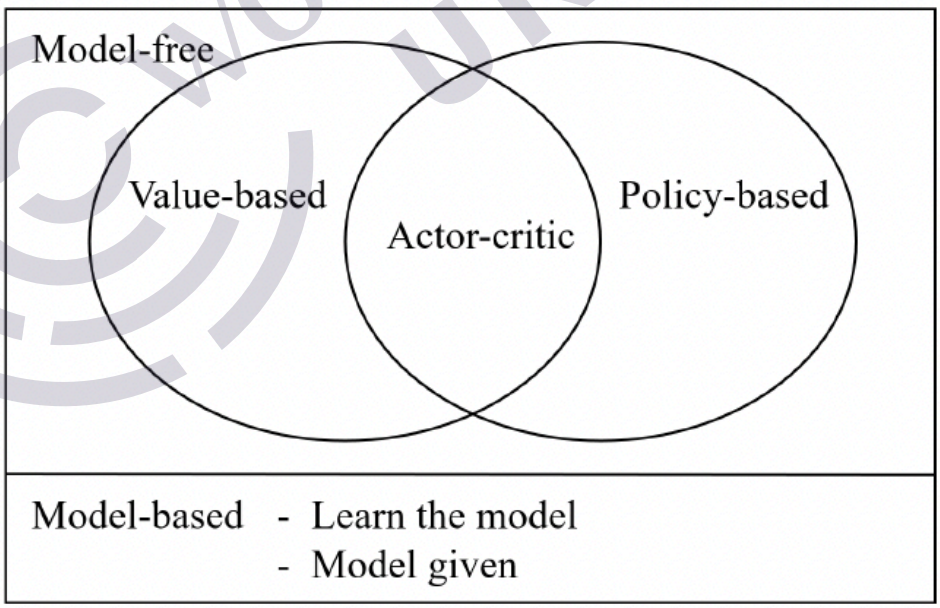
Note: If you use the "FinRL_StockTrading_Fundamental.ipynb" which uses fundamental indicators, try using the fundamental data from AlphaVantage, which we learned to do in Module 6 of the Financial Data course.

1. Preprocess the data: For example, including technical indicators and investor preferences like risk aversion
2. Design the environment: Define your state and action spaces, select your train/trade date split
3. Implement DRL algorithm: Choose your DRL algorithm and train the model.

2. The DRL Algorithms

The following two diagrams provide two different visualizations of the overlap and distinction between the deep reinforcement learning algorithms, given each algorithm's incorporation of policy and value estimation and whether the algorithm is model-free or model based. The first shows the overlap between the three model-free types of algorithms, while the second specifies the algorithms in relation to the approach. You can perhaps deduce that DDPG, TD3, and SAC belong to the Actor-critic type of model-free DRL algorithm (in the figure 1) since they all use both Q-learning and policy optimization (according to the second figure). We touched on these similarities and differences in lesson 3, but in this lesson, we make them more explicit since you will be experimenting with them with the help of the FinRL library.

Figure 1: Overlap across DRL Algorithms

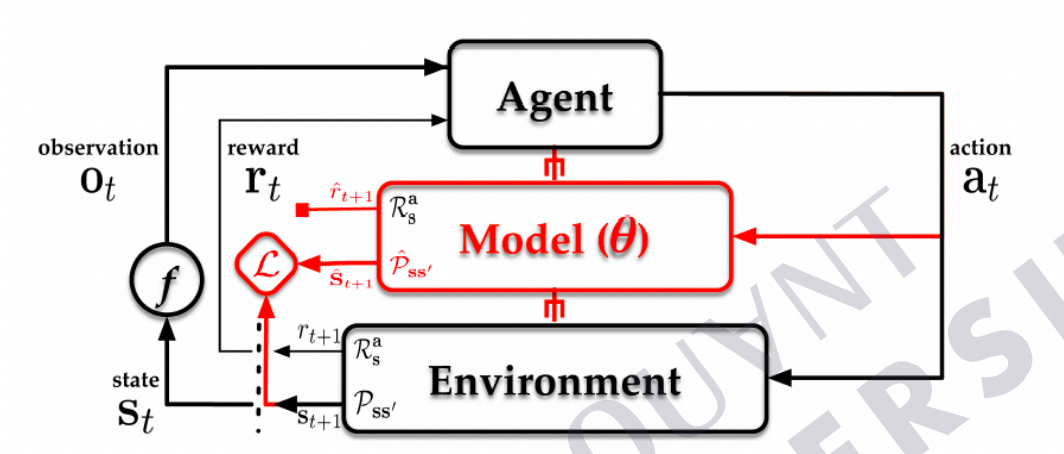


Source: Xiang, Xuanchen, and Simon Foo. "Recent Advances in Deep Reinforcement Learning Applications for Solving Partially Observable Markov Decision Processes (POMDP) Problems: Part 1—Fundamentals and Applications in Games, Robotics and Natural Language Processing." *Machine Learning & Knowledge Extraction*, 15 July 2021, https://www.mdpi.com/2504-4990/3/3/29.

2.1 Model-based vs. Model-free Types of Algorithms

Model-based algorithms require a fully known (Charpentier 23) or at least sufficiently knowable environment. In a deterministic environment, model-based algorithms can learn the environment. For this reason, Filos found that model-based algorithms performed very well in simulations involving significantly "well-behaving, easy-to-model and deterministic series" such as those exhibiting stable sinusoidal and sawtooth patterns (Filos 95). Another is dynamic programming, where the solutions of subproblems are stored and saved for later, rather than re-calculated each time; this information is then used to improve the predictiveness of the model, possibly by even informing a distribution for the predicted values. Since most financial markets are not known for being deterministic, you will notice that model-based algorithms do not appear much in the literature.

Figure 2: Model-based Reinforcement Learning

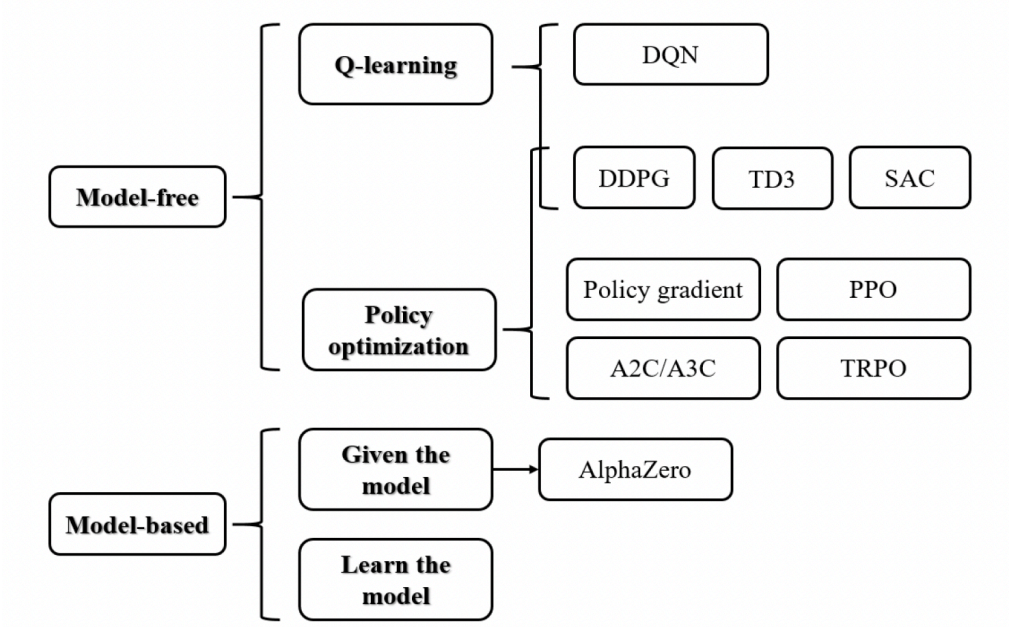


Source: Filos, Angelos. *Reinforcement Learning for Portfolio Management.* 20 Jun 2018. Imperial College London, MEng thesis. https://arxiv.org/pdf/1909.09571.pdf.

Model-free DRL algorithms, on the other hand, require neither an explicit model of the environmental dynamics nor an environment that behaves deterministically. Instead of constructing or parameterizing a model to inform the agent's action policy, we " parametrize the agent value function or/and policy directly" (Filos 67). Now, in order to do this, they rely on sample data, but they do not store solutions to specific subproblems to be reused later.

Model-free algorithms solve such equations as Bellman's optimality in an approximate way, without recourse to a distributional hypothesis of the investment returns or the like (Sato 8). Also known as "context-independent," these algorithms can better generalize across assets and markets, "regardless of the trading universe on which they have been trained" (Filos ii). The trade-off, however, is that they do require significant interaction time with the environment, e.g., to sample sufficiently, in order to achieve decent performance (Xiang 566, 568).

Figure 3: Family Tree of DRL Algorithms



Source: Xiang, Xuanchen, and Simon Foo. "Recent Advances in Deep Reinforcement Learning Applications for Solving Partially Observable Markov Decision Processes (POMDP) Problems: Part 1—Fundamentals and Applications in Games, Robotics and Natural Language Processing." *Machine Learning & Knowledge Extraction*, 15 July 2021, https://www.mdpi.com/2504-4990/3/3/29.

For concise and helpful descriptions of the specific algorithms referenced above (as well as useful code and additional reading), see here

- VPG (Vanilla Policy Gradient)
- DDPG (Deep Deterministic Policy Gradient)
- TD3 (Twin Delayed DDPG)
- PPO (Proximal Policy Optimization)
- SAC (Soft Actor-Critic)
- TRPO (Trust Region Policy Optimization)

2.2 Value-based vs. Policy-based Algorithms

As you recall from the last lesson, a policy maps a state to an action, and an optimal policy is a state-action mapping the results in the highest possible value. But there is not just one method for finding the optimal policy. In short, value-based methods seek to optimize the value functions first, and from these value functions, they derive an optimal policy; on the other hand, policy-based methods directly optimize the objective function for the policy (usually cumulative rewards), for example, via gradient descent (Li 30).

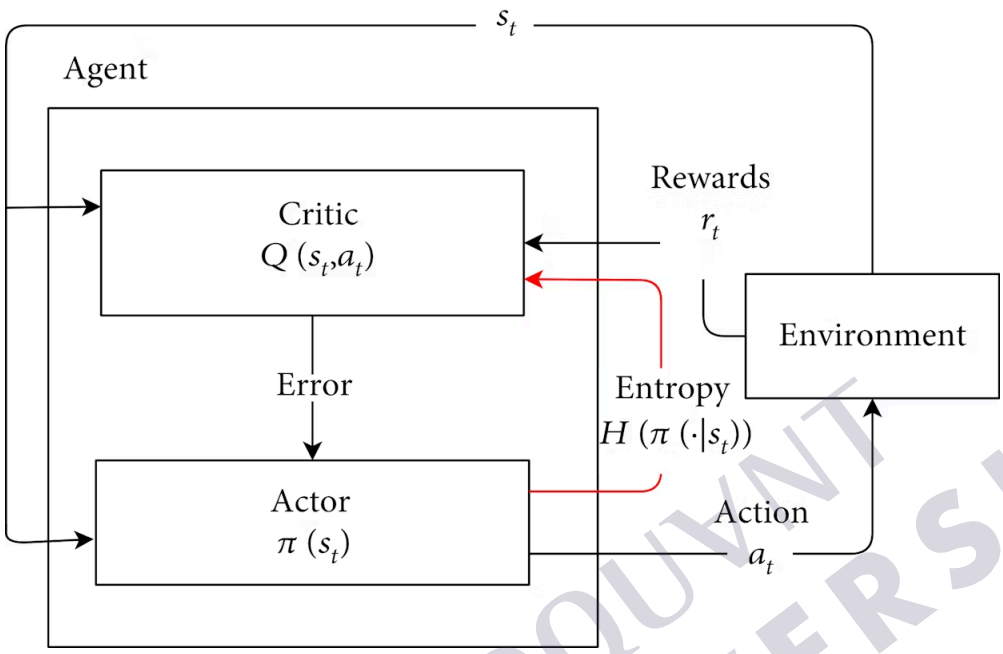
In contrast to value-based methods, policy-based methods optimize the policy (with function approximation) directly, and update the parameters by gradient ascent. Also, compared to value-based methods, policy-based methods usually have better convergence properties, are effective in high-dimensional or continuous action spaces, and can learn stochastic policies (Li 25), which is important for an agent in non-deterministic environments like most financial markets.

2.3 Actor-Critic Algorithms

As you can see in figure 1 above, Actor-Critic algorithms aim to take advantage of both Value and Policy approaches: "In actor-critic algorithms, the critic updates action-value function parameters, and the actor updates policy parameters, in the direction suggested by the critic" (Li 20).

In the **Soft Actor Critic** version of the algorithm, "the actor aims to maximize the expected reward and maximize the entropy simultaneously, such that it achieves fast learning with high randomness in the policy" (Zhiang 560). Hopefully, this recalls for you the maximum entropy principle that was discussed in Module 7, Lesson 3 of this course; here, you see it in action again. In figure 4 below, the red line indicates that the SAC agent tries to maximize entropy as well as expected discounted rewards—by adding an entropy term directly to the objective function (Ali 1).

Figure 4: The Actor-Critic Framework in the Reinforcement Learning Model



Source: Ali, Hamid et al. "Reducing Entropy Overestimation in Soft Actor Critic Using Dual Policy Network." *Wireless Communications and Mobile Computing*, 2021. https://www.hindawi.com/journals/wcmc/2021/9920591/.

Recognize that it may be more important to know the *relative* advantage of an action than the value itself: "The advantage function $A^{\pi}(s, a)$ corresponding to a policy π describes how much better it is to take a specific action a in state s, over randomly selecting an action according to $\pi(\cdot|s)$, assuming you act according to π forever after. Mathematically, the advantage function is defined by

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s).$$

This advantage function separates the **Advantage Actor Critic**, or A2C, from the vanilla actor-critic algorithm. But as Zhu points out, the *disadvantage* of the advantage function is a higher variance (5).

Asynchronous Advantage Actor Critic (A3C) is similar to its predecessor A2C except that it implements parallel training for multiple agents in parallel environments and updates a global value function (Xiang 560). As Zhu points out, this "greatly improves the data sampling rate of the on-policy algorithm and reduces the impact of poor samples (6).

3. Conclusion

In this lesson, we reinforced what we learned about reinforcement learning in the last lesson. We discussed some of the relevant theory in more detail, the best way to learn is by doing: So we implemented some of the most state-of-the-art reinforcement learning algorithms using an easy-to-use Python library called FinRL. We also read about the Frontier RL algorithm, which was

programmed using this library. It extends and modifies existing objective functions in practical and straightforward ways.

References

- AI4Finance Foundation. "FinRL/tutorials/1-Introduction/." GitHub. https://github.com/AI4Finance-Foundation/FinRL/tree/c34190153d84c376dcacaf18b57097a6272b0286/tutorials/1-Introduction.
- Ali, Hamid et al. "Reducing Entropy Overestimation in Soft Actor Critic Using Dual Policy Network." *Wireless Communications and Mobile Computing*, 2021. https://www.hindawi.com/journals/wcmc/2021/9920591/.
- Charpentier, Arthur et al. "Reinforcement Learning in Economics and Finance." _arXiv, _22 Mar 2020, https://arxiv.org/abs/2003.10014.
- Filos, Angelos. *Reinforcement Learning for Portfolio Management*. 20 Jun 2018. Imperial College London, MEng thesis. https://arxiv.org/pdf/1909.09571.pdf.
- Li, Yuxi. "Deep Reinforcement Learning." *arXiv*, 15 October 2018, https://arxiv.org/abs/1810.06339.
- Liu, Xiao-Yang, et al."FinRL: A Deep Reinforcement Learning Library for Automated Stock Trading in Quantitative Finance." *Arxiv*, 2 Mar 2022, https://arxiv.org/pdf/2011.09607.pdf.
- OpenAI. "Spinning Up." *GitHub*, https://github.com/openai/spinningup/tree/master/docs/algorithms.
- Pretorius, Ruan, and Terence van Zyl. "Deep Reinforcement Learning and Convex Mean-Variance Optimisation for Portfolio Management." TechRxiv, 22 Feb 2022, https://www.techrxiv.org/articles/preprint/Deep_Reinforcement_Learning_and_Convex_Mean-Variance_Optimisation_for_Portfolio_Management/19165745/1.
- Sato, Yoshiharu. "Model-Free Reinforcement Learning for Financial Portfolios: A Brief Survey." *arXiv*, 2019, p. 1–20, https://arxiv.org/abs/1904.04973.
- Xiang, Xuanchen, and Simon Foo. "Recent Advances in Deep Reinforcement Learning Applications for Solving Partially Observable Markov Decision Processes (POMDP) Problems: Part 1—Fundamentals and Applications in Games, Robotics and Natural Language Processing." *Machine Learning & Knowledge Extraction*, 15 July 2021, https://www.mdpi.com/2504-4990/3/3/29.
- Zhu, Jie et al. "An Overview of the Action Space for Deep Reinforcement Learning." *ACAI'21: 2021 4th International Conference on Algorithms, Computing and Artificial Intelligence,* Dec 2021, no. 52, pp 1–10, https://dl.acm.org/doi/abs/10.1145/3508546.3508598.